



Lab Report

Lab Report No : 01			
Lab Report Name : Missionaries and Cannibals Problem using DFS			
Course Title : Artificial Intelligence Sessional			
Course Code : CSE 422			
Name : MD Rakibul Hassan			
ID : 0822220105101024			
Level : 4	Term : 2	Section : A	Group : G1
Date of Submission : 19/04/2026	Semester : Spring	Year : 2026	

Marking Rubric

Problem Understanding & Report Clarity (3)	Implementation (4)	Results, Analysis & Presentation (3)	Total (10)

Key Learnings:

- Learned about the BFS and DFS algorithms with hands-on experience.
- Understand the working of the deque library and the append function.

Code Implementation:

```
DFS (Depth-First Search)
• It is a graph traversal algorithm used to explore all nodes in a graph.
• It explores as deep into the graph by visiting the children of the current node first, before moving back and exploring other neighbors.

# Missionaries and Cannibals Problem using DFS

# No need due to DFS uses a stack (list)
# from collections import deque

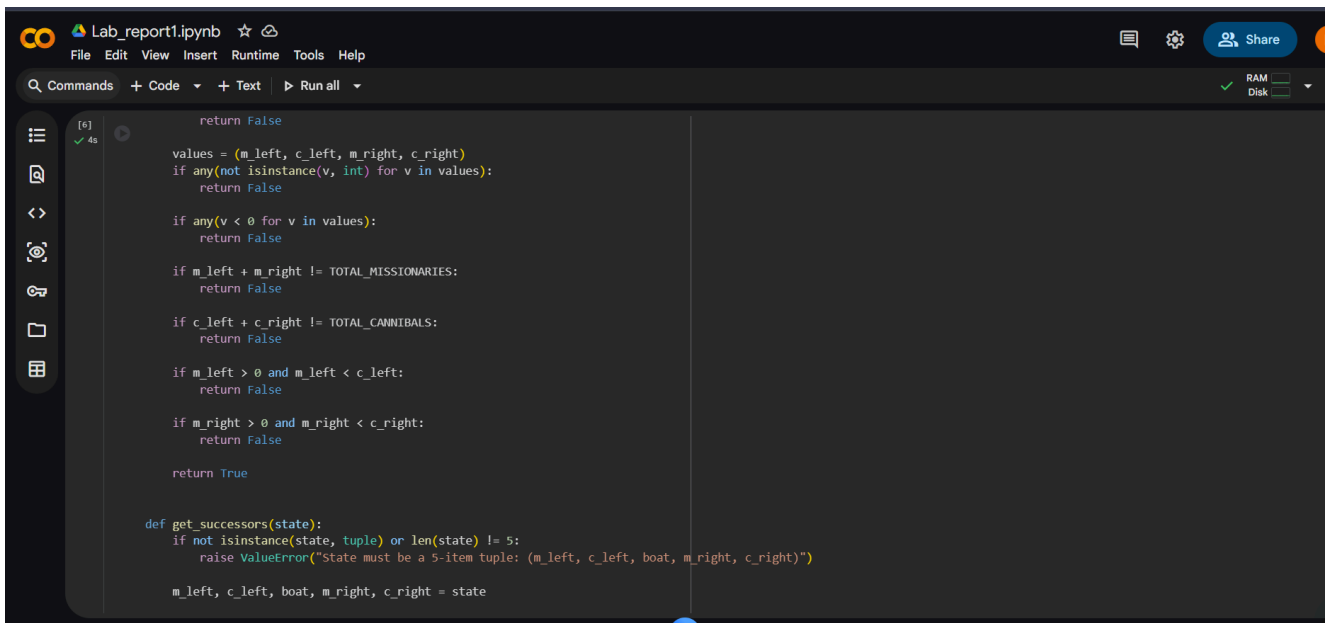
# Given Input
# TOTAL_MISSIONARIES = 3
# TOTAL_CANNIBALS = 3

# Input Flexibility
M = int(input("Enter the number of missionaries: "))
C = int(input("Enter the number of cannibals: "))

TOTAL_MISSIONARIES = M
TOTAL_CANNIBALS = C
VALID_BOAT_SIZES = {"L", "R"} # Boat capacity [2]
MOVES = ((1, 0), (2, 0), (0, 1), (0, 2), (1, 1))

# Dynamic Validation
def is_valid(m_left, c_left, boat, m_right, c_right):
    if boat not in VALID_BOAT_SIZES:
```

Figure No: 01



The image shows a Jupyter Notebook interface with a dark theme. The top bar includes the file name 'Lab_report1.ipynb' and standard menu options. The left sidebar contains icons for file operations. The main area displays a Python function that checks if a state is valid based on various constraints like missionaries, cannibals, and boat positions. The function returns True if the state is valid and False otherwise. A status bar at the bottom right shows 'RAM' and 'Disk' usage.

```
[6] ✓ 4s
return False

values = (m_left, c_left, m_right, c_right)
if any(not isinstance(v, int) for v in values):
    return False

if any(v < 0 for v in values):
    return False

if m_left + m_right != TOTAL_MISSIONARIES:
    return False

if c_left + c_right != TOTAL_CANNIBALS:
    return False

if m_left > 0 and m_left < c_left:
    return False

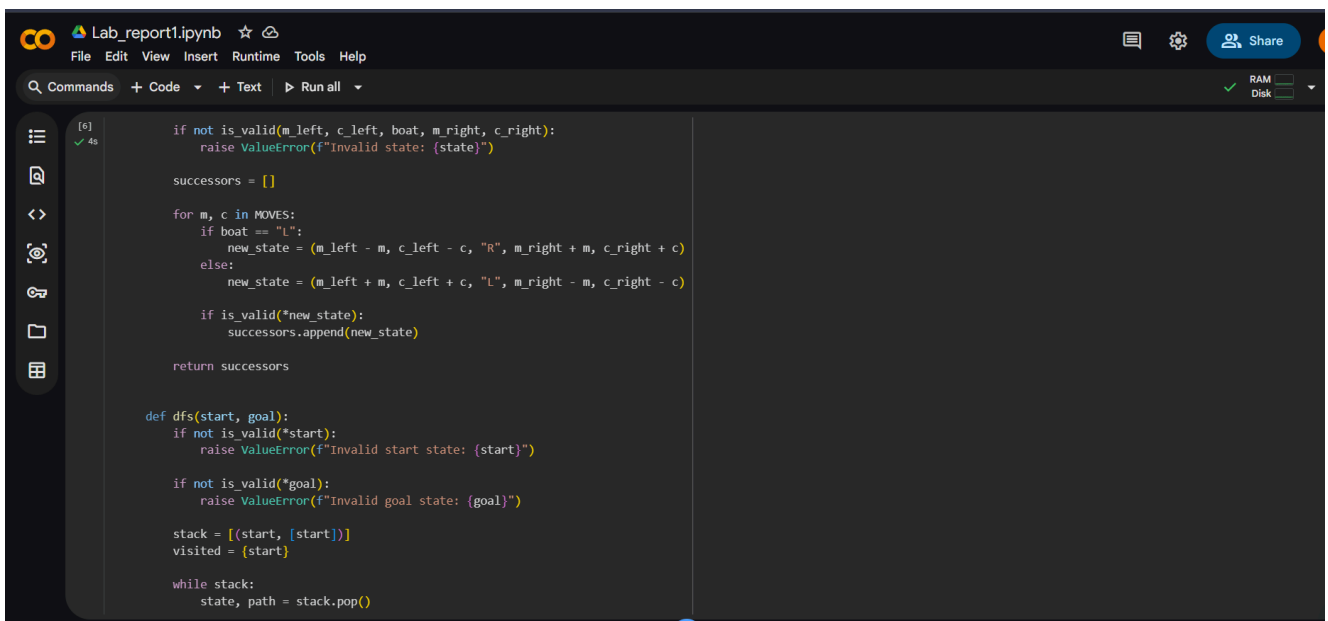
if m_right > 0 and m_right < c_right:
    return False

return True

def get_successors(state):
    if not isinstance(state, tuple) or len(state) != 5:
        raise ValueError("State must be a 5-item tuple: (m_left, c_left, boat, m_right, c_right)")

    m_left, c_left, boat, m_right, c_right = state
```

Figure No: 02



The image shows a Jupyter Notebook interface with a dark theme. The top bar includes the file name 'Lab_report1.ipynb' and standard menu options. The left sidebar contains icons for file operations. The main area displays a Python function that performs a Depth-First Search (DFS) on a state space. It checks for valid start and goal states, generates successors, and explores the state space using a stack. A status bar at the bottom right shows 'RAM' and 'Disk' usage.

```
[6] ✓ 4s
if not is_valid(m_left, c_left, boat, m_right, c_right):
    raise ValueError(f"Invalid state: {state}")

successors = []

for m, c in MOVES:
    if boat == "L":
        new_state = (m_left - m, c_left - c, "R", m_right + m, c_right + c)
    else:
        new_state = (m_left + m, c_left + c, "L", m_right - m, c_right - c)

    if is_valid(*new_state):
        successors.append(new_state)

return successors

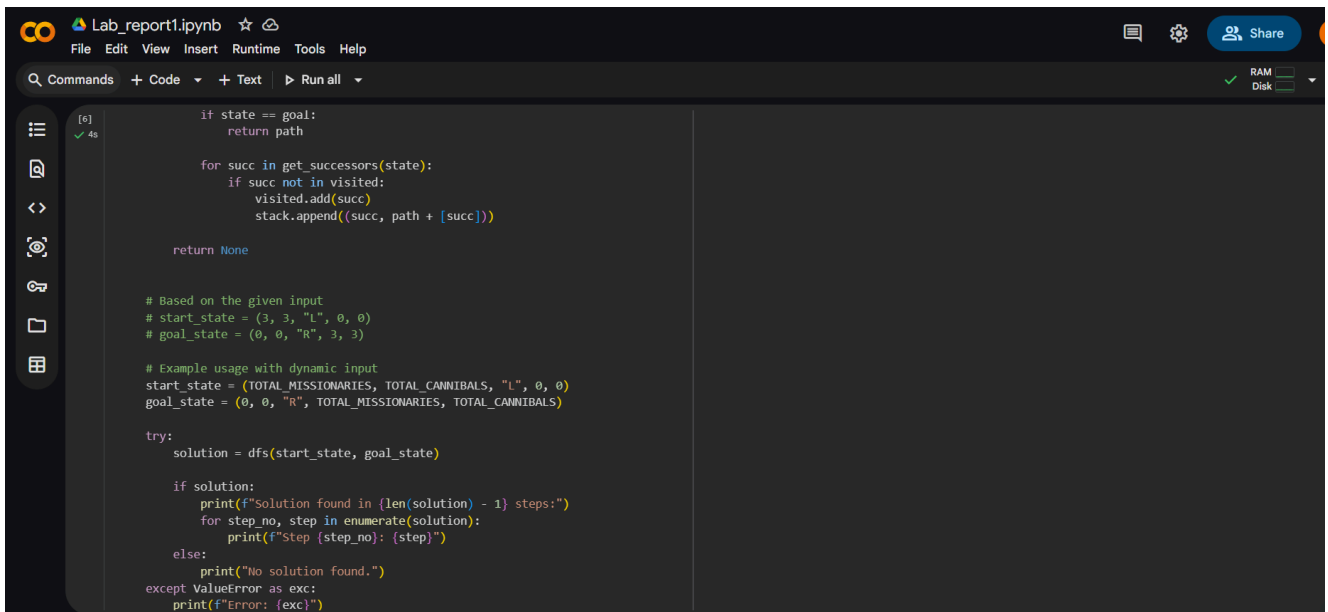
def dfs(start, goal):
    if not is_valid(*start):
        raise ValueError(f"Invalid start state: {start}")

    if not is_valid(*goal):
        raise ValueError(f"Invalid goal state: {goal}")

    stack = [(start, [start])]
    visited = {start}

    while stack:
        state, path = stack.pop()
```

Figure No: 03



```
if state == goal:
    return path

for succ in get_successors(state):
    if succ not in visited:
        visited.add(succ)
        stack.append((succ, path + [succ]))

return None

# Based on the given input
# start_state = (3, 3, "L", 0, 0)
# goal_state = (0, 0, "R", 3, 3)

# Example usage with dynamic input
start_state = (TOTAL_MISSIONARIES, TOTAL_CANNIBALS, "L", 0, 0)
goal_state = (0, 0, "R", TOTAL_MISSIONARIES, TOTAL_CANNIBALS)

try:
    solution = dfs(start_state, goal_state)

    if solution:
        print(f"Solution found in {len(solution) - 1} steps:")
        for step_no, step in enumerate(solution):
            print(f"Step {step_no}: {step}")
    else:
        print("No solution found.")
except ValueError as exc:
    print(f"Error: {exc}")
```

Figure No: 04

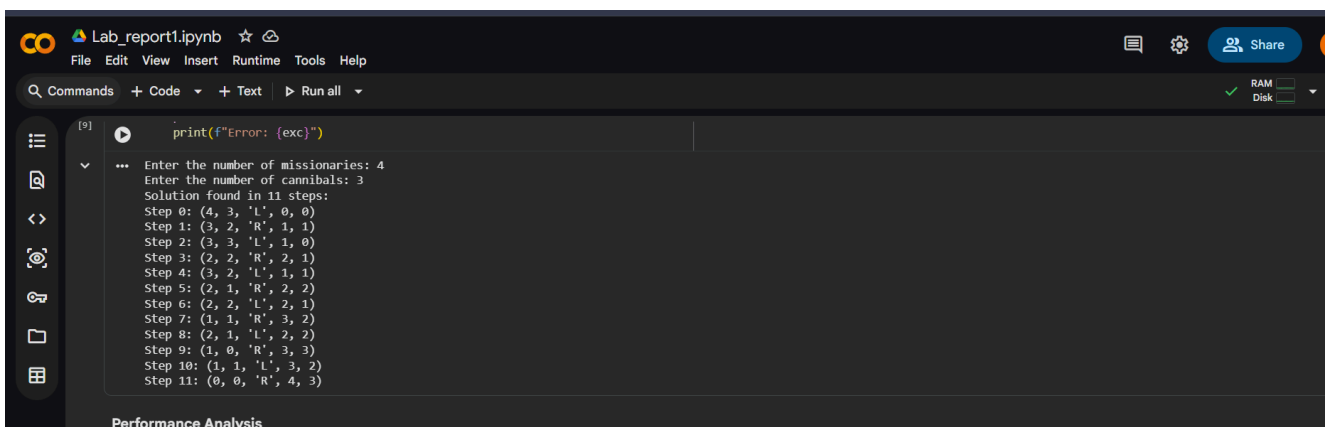
Sample Input, Output, and Result Analysis:

Input Sample:

Enter the number of missionaries: 4

Enter the number of cannibals: 3

Output Sample:



```
print(f"Error: {exc}")

... Enter the number of missionaries: 4
Enter the number of cannibals: 3
Solution found in 11 steps:
Step 0: (4, 3, 'L', 0, 0)
Step 1: (3, 2, 'R', 1, 1)
Step 2: (3, 3, 'L', 1, 0)
Step 3: (2, 2, 'R', 2, 1)
Step 4: (3, 2, 'L', 1, 1)
Step 5: (2, 1, 'R', 2, 2)
Step 6: (2, 2, 'L', 2, 1)
Step 7: (1, 1, 'R', 3, 2)
Step 8: (2, 1, 'L', 2, 2)
Step 9: (1, 0, 'R', 3, 3)
Step 10: (1, 1, 'L', 3, 2)
Step 11: (0, 0, 'R', 4, 3)
```

Figure No: 05

Result Analysis

- While DFS can solve the problem, BFS offers a more optimal solution for it.
- The total of 11 moves is needed for transferring all missionaries and cannibals to the right bank.
- DFS uses a stack or recursion.
- Follows all rules:
 - Boat position valid.
 - Non-negative values and totals remain constant.
 - Missionaries must outnumber or equal cannibals on both sides.